

Soluzione: Algoritmo per il problema EDIT-sequence-to-graph

1 Definizione del Problema e Sistema di Punteggio

L'obiettivo è trovare la distanza genomica minima tra un pattern P di lunghezza k e un cammino all'interno di un grafo diretto aciclico (DAG) $G=(V, E)$, dal nodo sorgente s al nodo terminale t . Il grafo possiede n vertici e m archi.

Sia $label(v)$ il carattere associato al vertice v . Definiamo la funzione di costo per la sostituzione $C(x, y)$ come segue:

$$C(x, y) = \begin{cases} 0 & \text{se } x = y = A \\ 1 & \text{se } x = y \text{ e } x \neq A \\ 4 & \text{se } x \neq y \end{cases} \quad (1)$$

Il costo per un'inserzione o una delezione è stabilito a $g = 4$.

2 Progettazione dell'Algoritmo

Link a implementazione python toy example:

<https://github.com/sammiforza3/Edit-sequence-to-graph>

Il problema viene risolto tramite un'estensione degli algoritmi di allineamento classico applicata ai grafi diretti aciclici, nota come Partial Order Alignment (POA). L'algoritmo utilizza la programmazione dinamica computando una matrice M di dimensioni $(k+1) \times (n+1)$, dove l'elemento $M[i, v]$ rappresenta il costo minimo per allineare il prefisso del pattern $P[1..i]$ con un cammino sul grafo che termina nel vertice v .

Essendo il grafo un DAG, è possibile stabilire un ordinamento topologico dei vertici. Questo garantisce che, nel calcolo del valore per un nodo v , i valori di tutti i suoi predecessori siano già stati calcolati.

2.1 Formula di Ricorrenza

Per calcolare il valore di una cella generica $M[i, v]$, l'algoritmo valuta il minimo tra tre possibili mosse:

$$M[i, v] = \min \begin{cases} \min_{u \in Pred(v)} \{M[i-1, u]\} + C(P[i], label(v)) & \text{(Sostituzione)} \\ M[i-1, v] + g & \text{(Inserzione nel pattern)} \\ \min_{u \in Pred(v)} \{M[i, u]\} + g & \text{(Delezione dal pattern)} \end{cases} \quad (2)$$

dove $Pred(v)$ rappresenta l'insieme dei vertici u tali per cui esiste un arco orientato da u a v e C la funzione di costo vista precedentemente.

3 Fasi dell'Algoritmo

1. Ordinamento Topologico: Si calcola un ordinamento topologico dei vertici V . Per gestire correttamente i gap iniziali, si introduce un nodo fittizio iniziale s_0 connesso alla vera sorgente s .
2. Inizializzazione:

- $M[0, s_0] = 0$

- Colonna iniziale: $M[i, 0] = i \cdot g$
 - Riga iniziale: $M[0, v] = \min_{u \in Pred(v)} \{M[0, u]\} + g$ seguendo l'ordinamento topologico.
3. Riempimento della Matrice: Si itera per ogni riga i (da 1 a k) e, all'interno di essa, per ogni vertice v secondo l'ordinamento topologico. Si applica la formula di ricorrenza salvando le scelte ottime in una matrice ausiliaria dei puntatori per il backtracking.
 4. Backtracking: Il costo della minima distanza genomica si troverà nella cella $M[k, t]$. Partendo da questa cella e seguendo i puntatori a ritroso fino a $[0,0]$, si ricostruisce l'elenco dei vertici del cammino ottimo.

4 Analisi del Costo Computazionale

4.1 Complessità Temporale

- Ordinamento Topologico: Richiede un tempo proporzionale all'esplorazione del grafo tramite DFS, ovvero $\mathcal{O}(n + m)$.
- Calcolo della Matrice: La matrice ha k righe e n colonne. Per calcolare una singola cella $M[i, v]$, è necessario scorrere tutti i predecessori del nodo v . La somma del grado entrante di tutti i vertici n in un grafo diretto è pari al numero totale degli archi m . Di conseguenza, processare un'intera riga i richiede un tempo proporzionale a $\mathcal{O}(m)$. Poiché l'operazione va ripetuta per ciascuna delle k righe del pattern, il costo totale per il riempimento della matrice è $\mathcal{O}(k \cdot m)$.

Il costo temporale complessivo è dominato dalla costruzione della matrice, risultando quindi in $\mathcal{O}(n + k \cdot m)$.

4.2 Complessità Spaziale

L'algoritmo richiede di allocare in memoria la matrice dei costi M di dimensioni k per n e una matrice di puntatori delle medesime dimensioni per consentire il backtracking. Il costo spaziale complessivo è pertanto $\mathcal{O}(k \cdot n)$.

4.3 Toy Example

Di seguito è riportato un semplice toy example per mostrare la costruzione della matrice, individuare la distanza di edit e evindeziare i nodi del grafo G che corrispondono alla stringa con distanza di edit minore rispetto al Pattern P .

$$\text{Pattern } P = CAC.$$

Visualizzazione del Grafo (Nodi e Lettere)

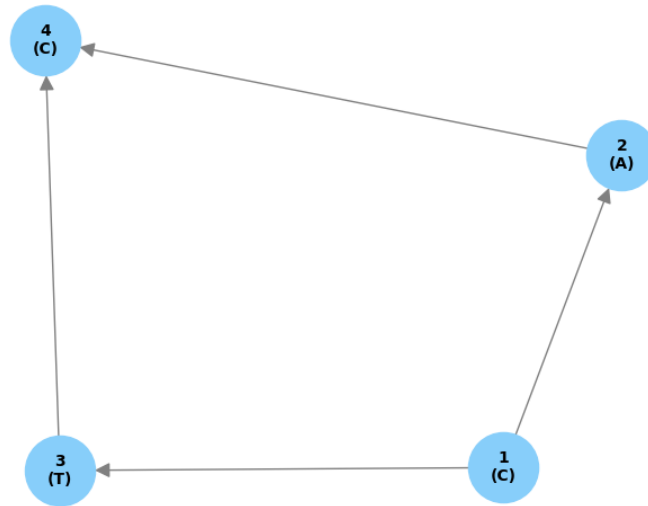


Figura 1: Grafo utilizzato per il Toy Example

Visualizzazione del Grafo (Nodi e Lettere)

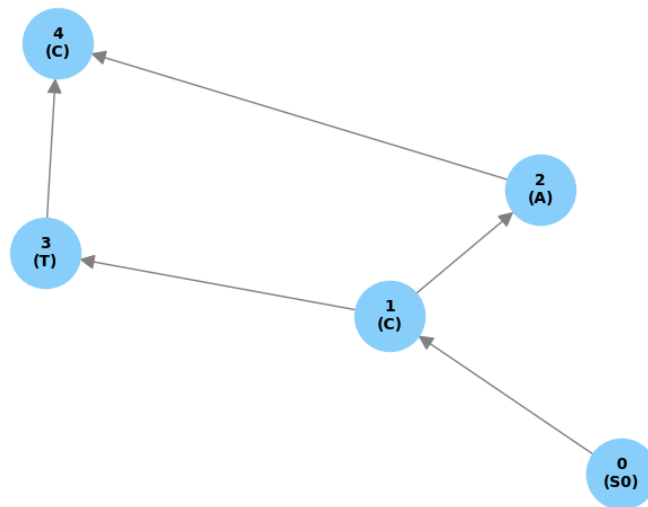


Figura 2: Grafo utilizzato per il Toy Example con nodo S_0 aggiunto

iniziamo a costruire la matrice ordinando i nodi topologicamente sulle colonne, il pattern andrà a occupare le righe, un carattere per riga.

	0	1	3	2	4
	<i>S0</i>	<i>C</i>	<i>T</i>	<i>A</i>	<i>C</i>
<i>e</i>					
<i>C</i>					
<i>A</i>					
<i>C</i>					

Tabella 1: Matrice delle distanze vuota

Avviamo ora i primi passaggi andando a inserire i casi più semplici.

- $M[0, 0] = 0$
- $M[i, 0] = i \cdot 4, i < k + 1$
- $M[0, v] = \min_{u \in Pred(v)} \{M[0, u] + g\}$

	0	1	3	2	4
	<i>S0</i>	<i>C</i>	<i>T</i>	<i>A</i>	<i>C</i>
<i>e</i>	0	4	8	8	12
<i>C</i>	4				
<i>A</i>	8				
<i>C</i>	12				

Tabella 2: Matrice delle distanze con i primi casi eseguiti

Eseguiamo ora i vari passaggi per riempire il resto delle celle

1. Cella C-1(C): è un match di costo 1 (caratteri diversi da 'A'), la scelta di costo minimo è eseguire un match prelevando il valore della cella $M[0, 0]$ che corrisponde al predecessore del nodo associato.
2. Cella C-3(T): è un mismatch, costo 4, la scelta di costo minore è delezione, selezionamo il predecessore minore $M[C, 3] = M[C, 1] + g$
3. Cella C-2(A): è un mismatch, costo 4, la scelta di costo minore è delezione, selezionamo il predecessore minore $M[C, 2] = M[C, 1] + g$
4. Cella C-4(C): match (costo 1) e delezione pareggiano. Costo minimo: $\min(\{M[e, 3], M[e, 2]\}) + 1 = 8 + 1 = 9$.
5. Cella A-1(C): mismatch (costo 4). Scelta minima, inserzione dall'alto: $M[C, 1] + g = 1 + 4 = 5$.
6. Cella A-3(T): mismatch (costo 4). Scelta minima, sostituzione dal predecessore 1: $M[C, 1] + 4 = 1 + 4 = 5$.
7. Cella A-2(A): match speciale (costo 0). Scelta minima, sostituzione dal predecessore 1: $M[C, 1] + 0 = 1$.
8. Cella A-4(C): mismatch (costo 4). Scelta minima, delezione dal predecessore 2: $M[A, 2] + g = 1 + 4 = 5$.
9. Cella C-1(C): match (costo 1). Scelta minima, sostituzione da *S0* (o inserzione): $M[A, 0] + 1 = 8 + 1 = 9$.

10. Cella C-3(T): mismatch (costo 4). Scelta minima, sostituzione da nodo 1 o inserzione: $\min = 9$.
11. Cella C-2(A): mismatch (costo 4). Scelta minima, inserzione dall'alto: $M[A, 2] + g = 1 + 4 = 5$.
12. Cella C-4(C): match (costo 1). Scelta minima, sostituzione dal predecessore 2: $M[A, 2] + 1 = 1 + 1 = 2$.

	0	1	3	2	4
	<i>S0</i>	<i>C</i>	<i>T</i>	<i>A</i>	<i>C</i>
<i>e</i>	0	4	8	8	12
<i>C</i>	4	1	5	5	9
<i>A</i>	8	5	5	1	5
<i>C</i>	12	9	9	5	2

Tabella 3: Matrice delle distanze di edit per il Toy Example

La distanza di edit minima tra il pattern e il grafo è presente nella cella $M[k + 1, t + 1]$.

4.4 Backtracking

Per effettuare il backtracking viene usata una seconda matrice. Tale matrice presenta in ogni cella $M[i, c]$ le coordinate della cella da cui è stato prelevato il valore ottimale per calcolare il costo di tale mossa.

	0	1	3	2	4
	<i>S0</i>	<i>C</i>	<i>T</i>	<i>A</i>	<i>C</i>
0 <i>e</i>	(0, 0)	(0, 0)	(0, 1)	(0, 2)	(0, 3)
1 <i>C</i>	(0, 0)	(0, 0)	(1, 1)	(1, 1)	(0, 2)
2 <i>A</i>	(1, 0)	(1, 1)	(1, 1)	(1, 1)	(2, 3)
3 <i>C</i>	(2, 0)	(2, 0)	(2, 1)	(2, 3)	(2, 3)

Tabella 4: Matrice di backtracking completa

Una volta costruita la matrice dei puntatori, è sufficiente ripercorrere il cammino a ritroso a partire dalla cella finale. Ogni cella visitata contiene le coordinate della cella precedente a cui è necessario saltare; l'etichetta della colonna corrispondente indica il vertice del grafo da inserire nel cammino ottimo.